STUDY GUIDE / VERSION 1.1

Big Data & Connectionist AI

The question

How does a neural network learn to recognize a handwritten digit from data? Follow one image from raw pixels to a prediction you can check.

Before you start

You will need basic arithmetic, decimals and a little algebra. You can do the activities with a pencil and calculator.

What you will learn

Check a dataset for duplicate or unreliable records. Calculate how a neuron responds to pixels. Explain how an optimizer uses gradients during training. Test a prediction on unseen handwriting and decide when a person should review it.

Contents

2 Big Data and data preparation
15 Connectionist AI and artificial neurons
24 How a neural network learns
31 Applications and responsible use
32 Conclusion
33 References and glossary
35 Practice activities
38 Worked answers and reflection

Suggested route

After Part 1, try activity A (p. 35). After Part 2, try B (p. 36). After Parts 3 and 4, finish C (p. 37). Compare your reasoning with the answers on p. 38.

As the creators of this material, we hope to make your learning journey clear, engaging, and rewarding. By the end, we wish you to have a solid foundation in Big Data and Connectionist AI, and a better understanding of how they work together to shape modern technology.

We wish you an enjoyable learning experience!

Part 1: Big Data

Every single time you use a digital device, small pieces of information are continuously created behind the scenes. For instance, when you buy a item online, send a quick message to a friend, or check a map location on your phone, a computer records that exact moment.

Naturally, a single record by itself takes up almost no space at all on a computer drive. However, when millions of people do these actions repeatedly every minute, the total amount of stored information grows extremely fast into enormous collections.

Consequently, standard home computers struggle greatly to handle these huge collections of information. When a dataset reaches a point where traditional tools can no longer store or process it smoothly, experts call this phenomenon **Big Data**.

> **DEFINITION — Big Data:** Big Data is a massive collection of information whose overall size, speed of arrival, and varied forms require special methods and modern tools to store, process, and understand effectively.

Importantly, Big Data is not simply about having a huge collection of files on a disk. Instead, it represents a practical challenge of working with information fast enough so that people can actually use it before it becomes old.

To make this idea easier to imagine, think of a massive library called the Library of ALEKSANDRIA. If the library gets a few new books every week, the librarians can easily sort them on wooden shelves by hand.

But continuously imagine that thousands of new papers arrive at the front door every second of the day. Furthermore, these papers come in hundreds of different languages, some are damaged, and others contain simple writing mistakes that need fixing.

Clearly, the main challenge for the library is no longer just finding enough physical shelf room for the paper. More importantly, the librarians must work out how to organize, check, and find useful information quickly enough to answer questions.

Similarly, modern computing systems face this exact issue every day. Fortunately, experts break this grand challenge down using five distinct properties, which are widely known today as the **5 V's of Big Data**.

> **NOTE — The Core Framework:** The 5 V's are five clear lenses that help us examine a data system: Volume, Velocity, Variety, Veracity, and Value. They work together to show us where the biggest technical obstacles lie.

First of all, we look closely at **Volume**, which simply means the physical quantity of stored items. When systems collect billions of separate logs, storage space must expand rapidly to keep up with the physical demand.

Secondly, we encounter **Velocity**, which measures how fast new information is generated and transferred across networks. In many modern systems, data arrives constantly like a fast river rather than a slow dripping tap.

Thirdly, we must manage **Variety**, which refers to the many different structural forms that information takes. Modern systems must handle neat text tables alongside raw voice recordings, photos, and high-definition video clips seamlessly.

Fourthly, we have to inspect **Veracity**, which measures the truthfulness and reliability of the incoming records. Sadly, real-world data often contains typing errors, missing parts, or duplicate entries that can easily confuse an analyst.

Finally, we arrive at **Value**, which asks whether all this effort actually produces a useful result. Storing petabytes of raw numbers brings no real benefit unless it helps people solve real problems or make smarter decisions.

The five Vs of Big Data

Volume	Velocity	Variety	Veracity	Value
How much?	How fast?	Which formats?	How reliable?	What benefit?

Now, let us explore how data actually moves from its original source into a computer system step by step. This entire journey starts at the exact moment a digital event happens somewhere in the world.

Initially, an electronic device like a thermometer sensor or a phone creates a raw measurement. Immediately after that, network cables or radio signals transfer this reading across space to a waiting computer.

Once received, the destination computer carefully saves the reading onto physical storage hardware. Afterward, specialized software steps in to organize, check, and clean the stored records so that they are ready for analysis.

During this intake process, systems encounter three main layout styles of data: **structured**, **semi-structured**, and **unstructured** data. Understanding these forms helps us choose the right tools for the job.

DEFINITION — Data Formats:

- **Structured Data:** Highly organized information that fits neatly into predictable tables with fixed rows and columns.
- **Semi-Structured Data:** Information that uses tags or markers to separate fields without using a fixed table.
- **Unstructured Data:** Raw information that lacks any regular pattern or predefined schema, such as images or audio.

For instance, structured data looks like a clean spreadsheet where every row has an exact customer ID number and a purchase price. Because everything sits in neat columns, computers can search through it extremely fast.

Meanwhile, semi-structured data uses simple tags or labels to organize facts flexibly. A common example is a digital text document where labels clearly mark where a person's name starts and ends.

On the other hand, unstructured data includes items like raw digital photographs, voice recordings, and video clips. These files do not have built-in rows or columns, so software must work much harder to find patterns inside them.

Besides handling different structures, computing systems must also decide how fast to process incoming files. Depending on how quickly an answer is needed, engineers choose between **batch processing** and **stream processing**.

DEFINITION — Processing Methods:

- **Batch Processing:** Collecting a large group of data records over time and processing them all together as one big job.
- **Stream Processing:** Checking and calculating incoming data records continuously one by one as soon as they arrive.

To understand batch processing, imagine our library collecting returned books all day in a big wooden bin. At midnight, when the doors close, the staff sorts all the collected books together in one planned session.

Conversely, stream processing is like a fire alarm monitor in the library hallway. The moment a smoke sensor detects a sudden temperature jump, the system must process that signal immediately without waiting for midnight.

Therefore, online shops might process their daily sales totals in nightly batches to update financial ledgers. At the same time, security tools use fast stream processing to block fraudulent credit card transactions instantly.

However, whether data arrives in fast streams or quiet batches, we cannot simply assume that every incoming record is correct. This brings us directly to the vital concept of data quality and **veracity**.

Two ways to process event records

Batch	Collect records	→	Process a group	→	Daily totals
Stream	Receive an event	→	Process on arrival	→	Live alert

Real-world measurements are rarely perfect right out of the box. Frequently, human operators make typing mistakes, sensors suffer from brief electrical faults, or network connections drop out mid-sentence.

Consequently, uncleaned data can easily lead to incorrect conclusions if we process it blindly. Therefore, engineers run incoming files through a strict multi-step cleaning pipeline before doing any serious analysis.

First, **data validation** checks if new entries follow basic rules. For example, a validation rule checks whether a user entered a valid calendar date rather than random letters into a birthdate box.

Next, **data filtering** systematically removes unwanted or completely broken records from the collection. If a temperature sensor accidentally sends a reading of minus five hundred degrees, the filter safely drops it.

After filtering, **deduplication** searches for identical records that were accidentally saved twice. By removing exact copies, the system prevents numbers from being counted twice in final totals.

Subsequently, software handles missing values by filling empty gaps with reasonable estimates or marking them clearly as unknown. Finally, **data transformation** converts different measurement units into one standard format.

DEFINITION — Data Preparation Terms:

- **Validation:** Verifying that data follows strict logical rules and structural rules.
- **Filtering:** Removing records that are irrelevant or fail basic safety checks.
- **Transformation:** Changing data formats or units into one standard style for easier reading.

Even after careful cleaning, a dataset can still suffer from a subtle problem called **bias**. Bias happens when a dataset systematically favors certain groups or outcomes while missing others completely.

For example, if a library conducts a feedback survey exclusively using a mobile phone application, it will likely miss older visitors who prefer paper. Even if all application answers are perfectly cleaned, the overall result stays biased.

Crucially, automatically cleaning data with software cannot remove underlying bias by itself. Thus, human specialists must always think carefully about how data was gathered in the first place.

Now that we understand how data is cleaned, we should examine the physical hardware used to store these massive collections. Information always lives on physical hardware, starting from small drives up to global facilities.

At the smallest scale, computers record information on local storage drives. The two most common types used today are **Hard Disk Drives (HDDs)** and **Solid-State Drives (SSDs)**.

DEFINITION — Storage Hardware:

- **HDD (Hard Disk Drive):** A storage device that saves data magnetically on spinning physical platters.
- **SSD (Solid-State Drive):** A fast storage device that saves data in electronic flash memory without moving parts.

Mechanically, HDDs rely on spinning magnetic disks and moving read heads to retrieve files. Because they have physical moving parts, HDDs are somewhat slower, but they offer huge storage space for a low price.

In contrast, SSDs use flash memory chips with no moving parts at all. As a result, SSDs read and write files much faster than HDDs, making them ideal for tasks that need instant access to data.

Besides speed, engineers must balance two other key features: **capacity** and **endurance**. Capacity tells us how much total data a drive holds, while endurance measures how many total writes a drive survives over time.

Inside a hard disk and a solid-state drive



HDD

Moving head reads a spinning platter.



SSD

Flash chips store bits electronically.

When an organization grows beyond a single computer drive, it starts using dedicated server hardware. A **server** is simply a powerful computer that provides shared files or services to other computers over a network.

The computers that request services from a server are called **clients**. For instance, when you browse a library catalog from your laptop, your laptop acts as the client, while the central system acts as the server.

DEFINITION — Server & Client:

- **Server:** A centralized computer dedicated to storing files and providing services across a network.
- **Client:** A computer or application that connects to a server to request files or run tasks.

As companies add more servers to handle growing workloads, they mount them neatly inside standardized metal frames called **server racks**. Racks keep power cables, network wires, and cooling paths neatly organized.

Eventually, when many server racks fill an entire room, that space becomes a dedicated **server room**. Managing a server room requires powerful air conditioners to prevent equipment from overheating during heavy use.

At the absolute largest scale, organizations build entire specialized buildings called **data centres**. A modern data centre houses thousands of active servers alongside massive backup generators, cooling towers, and strict security guards.



Naturally, measuring storage space across these massive data centres requires huge numerical units. Computer memory builds up in steps of roughly one thousand, starting from the basic single **byte**.

NOTE — Scale Units of Storage:
- **1 Kilobyte (KB):** ~1,000 bytes (a short text note)
- **1 Megabyte (MB):** ~1,000 KB (a digital photo)
- **1 Gigabyte (GB):** ~1,000 MB (a high-definition movie)
- **1 Terabyte (TB):** ~1,000 GB (a large home hard drive)
- **1 Petabyte (PB):** ~1,000 TB (a large corporate archive)
- **1 Exabyte (EB):** ~1,000 PB (massive global data flows)

To link these scattered servers together smoothly, organizations rely on computer networks. Depending on how far the network spreads geographically, experts group networks into three main categories: LAN, MAN, and WAN.

DEFINITION — Network Categories:
- **LAN (Local Area Network):** A network connecting devices within a small area like a single office or building.
- **MAN (Metropolitan Area Network):** A high-speed network spanning an entire city or large town.
- **WAN (Wide Area Network):** A broad network spanning long distances across countries or continents.

For example, all computers inside our single library building connect together using a local LAN network. This allows staff to share local printers and access central catalog files quickly across rooms.

Furthermore, if the library system links ten branch libraries located across the same city, it uses a MAN network. Finally, connecting to public systems across the entire world requires the global reach of a WAN network, such as the internet.

Within a local network, businesses often attach dedicated storage boxes called **NAS (Network Attached Storage)**. A NAS box lets multiple workers save and open shared office files securely over the local network connection.

Alternatively, many modern organizations choose to use **cloud storage** services instead of buying physical drives. Cloud storage means paying an external provider to host your files on their remote internet-connected data centres.

DEFINITION — NAS & Cloud Storage:

- **NAS:** Dedicated file storage hardware connected directly to a local network for shared access.
- **Cloud Storage:** Remote storage space hosted by a third-party company and accessed securely over the internet.

Using cloud storage is very much like renting shelf space in a professional archive warehouse instead of building your own storage room. It saves equipment costs, but you must still ensure that your remote files remain private and secure.

When companies buy or sell these technology services, they operate under two main commercial business models: **B2B** and **B2C**.

DEFINITION — Business Models:

- **B2B (Business-to-Business):** A commercial setup where one company sells products or tech services to another company.
- **B2C (Business-to-Consumer):** A commercial setup where a company sells products or services directly to individual people.

For example, a cloud data centre selling massive server storage to an online bank is a classic B2B relationship. Conversely, when that same bank offers a mobile savings app directly to everyday users, it operates a B2C model.

Now that we have covered collection, storage, and networking, we can finally explore **data analysis**. This is the crucial step where prepared numbers are turned into meaningful insights that support decisions.

Generally speaking, a complete data analysis project follows seven clear steps in order: defining a question, gathering data, preparing files, processing numbers, analyzing patterns, visualizing findings, and evaluating results.

First, **data processing** transforms raw measurements into an organized state by sorting rows, calculating sums, or grouping categories together cleanly.

DEFINITION — Data Processing & Analysis:

- **Data Processing:** Executing automated actions on raw data to organize, clean, or format it.
- **Data Analysis:** Examining processed data systematically to uncover patterns, test ideas, and answer questions.

Next, analysts apply **data analysis** techniques to discover what the numbers actually mean. Depending on the primary question being asked, analysis falls into four major categories.

To start with, **descriptive analysis** answers "What happened?" by summarizing historical records, such as calculating how many total books were borrowed during the past month.

Next, **diagnostic analysis** asks "Why did it happen?" by looking for root causes, such as discovering that book loans dropped because the main street was closed for repairs.

Looking forward, **predictive analysis** asks "What will likely happen next?" by using historical trends to estimate future demand, such as forecasting next month's library visitor numbers.

Finally, **prescriptive analysis** suggests "What action should we take?" by recommending smart choices, such as advising the library manager to order extra study tables before exam week begins.

Four questions an analyst can answer

Descriptive	Diagnostic	Predictive	Prescriptive
What happened?	Why did it happen?	What is likely next?	What should we do?

To share these analytical findings clearly with non-technical decision-makers, specialists use **data visualization**. Visual charts translate walls of boring numbers into clear graphical pictures that anyone can read easily.

For example, a simple **bar chart** works wonderfully when comparing total quantities across separate groups. Similarly, a continuous **line chart** easily shows how a measurement rises or falls over several months.

Additionally, a **scatter plot** helps reveal hidden relationships between two different numerical variables, while a **map** shows geographic patterns across different real-world locations.

> **DEFINITION — Data Visualisation:** The graphical representation of statistical data and analytical findings using visual elements like charts, graphs, maps, or interactive dashboards.

However, visual charts can sometimes deceive viewers if they are drawn poorly. For instance, chopping off the bottom of a chart scale can make a tiny two percent increase look like a massive jump, so clarity matters.

Beyond business dashboards, Big Data analysis plays a vital role across modern society today. Transport networks study live GPS logs to reduce traffic jams, while medical researchers analyze patient records to spot disease trends early.

Nevertheless, storing and analyzing massive amounts of personal information brings up serious **ethical questions**. Organizations must carefully consider how their technological systems impact human lives and personal freedom.

Crucially, **data privacy** ensures that personal information is collected fairly and protected from unauthorized viewing. Organizations should practice **data minimisation** by gathering only the data they truly need for a specific task.

DEFINITION — Ethical Guidelines:

- **Data Privacy:** Protecting personal information and allowing individuals control over their data.
- **Data Security:** Technical protections guarding systems against unauthorized access, loss, or damage.
- **Data Minimisation:** The principle of collecting and keeping only the data necessary for a task.

Furthermore, companies must maintain strong **data security** systems like passwords and encryption to prevent harmful security breaches. They must also obtain **informed consent** by explaining data use clearly to users.

Ultimately, mastering Big Data involves much more than just building huge server racks and writing fast code. True success requires collecting clean data responsibly, analyzing it accurately, and using it ethically to make the world better.

Parts 2 & 3: Connectionist AI & Neural Network Learning

Having explored how massive quantities of data are gathered, cleaned, and stored in Part 1, we now transition naturally to understanding how intelligent systems process that data. In this double section, we explore **Connectionist AI**, artificial neural network architectures, and the exact mechanics of how these networks learn from raw numbers.

Part 2: Connectionist AI & Network Architecture

Traditionally, classic computer programs relied on human coders writing out long lists of explicit, step-by-step instructions. For example, if a temperature sensor reads above a certain threshold, a simple program turns on a cooling fan automatically.

However, many real-world tasks cannot be solved using simple static rules. Recognizing a handwritten digit, spotting an object in a fuzzy photo, or interpreting spoken words involves thousands of tiny variations that are impossible to write out by hand.

To overcome this limitation, scientists created **Connectionist AI**. Instead of relying on hardcoded human rules, connectionist systems use large networks of simple computational units connected together to learn complex patterns directly from data.

> **DEFINITION — Connectionism:** An approach to modeling information processing through networks of simple, interconnected computational units whose combined mathematical activity produces complex behavior.

Generally speaking, Connectionist AI forms a major subfield within the broader landscape of Artificial Intelligence and Machine Learning. Understanding how these terms fit together helps clarify how modern software operates.

Artificial Intelligence (AI) serves as the broad overarching field dedicated to building machines capable of performing smart tasks. Within AI, **Machine Learning (ML)** focus on systems that improve their performance by analyzing data rather than following fixed hand-coded rules.

Finally, **Deep Learning** represents a specific branch of machine learning that utilizes artificial neural networks with multiple processing layers. Thus, deep learning sits inside machine learning, which in turn sits inside artificial intelligence.

> **DEFINITION — Artificial Intelligence (AI):** The general computer science field focused on creating systems capable of performing tasks that usually require human intelligence.

> **DEFINITION — Machine Learning (ML):** An approach where software uses mathematical patterns in data to improve predictions without being explicitly programmed for every scenario.

> **DEFINITION — Deep Learning:** A specialized subfield of machine learning that uses multi-layered artificial neural networks to learn deep abstractions from complex data.

Historically, researchers divided artificial intelligence into two major paradigms: symbolic approaches and connectionist approaches. Comparing them reveals why modern neural networks have become so popular for perceptual tasks.

Specifically, **Symbolic AI** represents knowledge using formal logic, human-written symbols, and explicit relational statements. For example, an ancient library system might use a strict cataloging rule stating that all documents published before 1900 belong to the historical archive.

In contrast, a connectionist system does not store knowledge as human-readable sentences or explicit logic lists. Instead, it absorbs thousands of examples and distributes its knowledge across numerical weights spread throughout a vast network.

DEFINITION — Symbolic AI: An approach to AI that explicitly represents knowledge using logical rules, human-crafted symbols, and formal relational statements.

DEFINITION — Artificial Neural Network (ANN): A computational model made of interconnected artificial neurons that transform continuous numerical inputs into useful output predictions.

To picture this distinction clearly, imagine two different ways to organize our Library of ALEKSANDRIA. A symbolic approach uses a master rulebook written by senior librarians to sort incoming papers into categories.

Conversely, a connectionist approach lets a team of assistants examine thousands of previously sorted books. Over time, the team discovers subtle mathematical patterns in text word counts and automatically categorizes new documents accurately.

Fundamentally, artificial neural networks draw loose inspiration from biological nervous systems found in nature. In the human brain, biological neurons receive electrical signals through dendrites, process them, and transmit signals onward through axons.

However, computer scientists stress that artificial neural networks are purely mathematical models. They do not simulate complex chemical reactions or biological cell structures; they simply perform numerical calculations fast.

DEFINITION — Biological Neuron: A specialized living cell in a biological nervous system that receives, processes, and transmits electrochemical signals.

DEFINITION — Artificial Neuron: A mathematical processing unit that combines numerical inputs, applies weight parameters and a bias, and produces an output value.

Inside a single artificial neuron, incoming data undergoes a straightforward sequence of arithmetic steps. First, every numerical input is multiplied by a specific number called a **weight**.

After multiplying inputs by their respective weights, the neuron adds all these numbers together to form a total weighted sum. Next, it adds an adjustable offset value called a **bias** to shift the calculated sum up or down.

Finally, this resulting sum passes through an **activation function**, which shapes the final numeric signal that the neuron sends forward to other units or the final output display.

DEFINITION — Input: A numerical value supplied to a neuron, representing either a raw data feature or the output score from an earlier processing unit.

DEFINITION — Weight: An adjustable numerical parameter that controls the strength and influence of an input connection on a neuron's total calculation.

DEFINITION — Bias: An adjustable numeric parameter added to the weighted sum of a neuron's inputs before applying the activation function.

DEFINITION — Weighted Sum: The total calculated value obtained by multiplying every input value by its paired weight multiplier and adding the results together.

DEFINITION — Output: The final numeric signal produced by an artificial neuron after passing its biased weighted sum through an activation function.



Crucially, activation functions play a vital structural role because they introduce **non-linearity** into the network system. Without non-linear activation functions, combining multiple processing layers would mathematically collapse into a simple single linear equation.

To prevent this collapse, engineers use several popular activation functions. For instance, the **ReLU (Rectified Linear Unit)** function turns any negative number into zero while keeping positive numbers unchanged.

Alternatively, the **Sigmoid** function squeezes any input value into a smooth decimal score between zero and one, making it ideal for estimating simple true-or-false probabilities.

Additionally, the **Softmax** function takes a list of raw output numbers from multiple neurons and scales them into a clean set of relative percentages that sum up to exactly one hundred percent.

DEFINITION — Activation Function: A mathematical function applied to a neuron's combined input sum to determine its final output signal.

DEFINITION — Non-Linearity: A mathematical property enabling a network to represent complex, curved boundaries and intricate relationships beyond simple straight lines.

NOTE — Common Activation Functions:
• **ReLU:** Outputs zero for negative values and passes positive values straight through.
• **Sigmoid:** Maps continuous input numbers into a tight decimal range between 0 and 1.
• **Softmax:** Converts a group of raw categorical scores into a valid probability distribution summing to 1.0.

Now, let us zoom out from an individual neuron to explore how thousands of neurons assemble into structured architectures. Neurons are typically organized in sequential functional groups called **layers**.

The very first layer in any architecture is the **input layer**, which receives raw numerical values directly from preprocessed data files, such as pixel brightness measurements from an image.

Following the input layer, data moves through one or more internal processing stages called **hidden layers**. These layers extract increasingly sophisticated patterns from intermediate calculations.

Finally, information arrives at the **output layer**, which produces the model's ultimate result, such as a category label or a numerical prediction score.

DEFINITION — Architecture: The overall structural design of a neural network, specifying its layers, connections, and internal signal paths.

DEFINITION — Input Layer: The initial processing layer of a network that ingests raw feature vectors directly from input datasets.

DEFINITION — Hidden Layer: An intermediate layer situated between input and output layers that transforms internal numerical representations.

DEFINITION — Output Layer: The final layer of a neural network that emits calculated predictions, probabilities, or target scores.

DEFINITION — Deep Neural Network: An artificial neural network containing multiple hidden layers between its input and output boundaries.

To visualize this flow, imagine a document passing through several workstation desks inside our Library. The receiving desk takes in the document, middle workstations translate and index chapters, and the final desk places an official catalog stamp on it.

In a standard **feedforward neural network**, numerical signals travel strictly in one forward direction from the input layer through hidden layers to the output layer without any back-stepping loops.

DEFINITION — Feedforward Neural Network: An architecture in which information flows unidirectionally forward from inputs to outputs without circular feedback loops.

DEFINITION — Representation: The structured numerical vector format used internally by a neural network to encode real-world objects or properties.

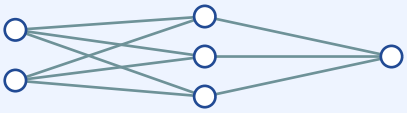
Different tasks demand different structural designs. For example, a basic **Multilayer Perceptron (MLP)** consists of fully connected layers where every neuron connects directly to every single unit in the next layer.

While MLPs work wonderfully for structured tabular data, they struggle when handling complex images because raw high-resolution photographs contain millions of spatial pixels that create far too many connections.

DEFINITION — Multilayer Perceptron (MLP): A classic feedforward neural network made of multiple densely connected processing layers.

DEFINITION — Fully Connected Layer: A neural network layer where every processing unit receives direct input connections from every single unit in the preceding layer.

A fully connected network



Inputs / hidden layer / output

Each line carries a learned weight.

To handle visual images efficiently, computer scientists developed **Convolutional Neural Networks (CNNs)**. Instead of connecting every pixel to every neuron, CNNs slide small numerical matrices called **filters** across spatial grids to detect local edges, textures, and shapes.

By reusing the exact same filter values across different image regions through an operation called **convolution**, CNNs drastically reduce parameter counts while preserving spatial geometric patterns.

DEFINITION — Convolutional Neural Network (CNN): A specialized network architecture designed to process spatial grid data like images by applying sliding mathematical filters.

DEFINITION — Convolution: A mathematical operation where a small numerical filter slides across a spatial grid to extract localized feature patterns.

DEFINITION — Filter (Kernel): A small matrix of adjustable numerical parameters applied across an input grid during convolution operations.

On the other hand, when processing sequential data unfolding over time—such as spoken speech recordings or sentences—standard feedforward networks struggle because they lack temporal memory.

To overcome sequence problems, engineers use **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** networks. These models maintain internal states that pass information across sequence steps.

DEFINITION — Recurrent Neural Network (RNN): A network architecture utilizing feedback state loops to carry temporal context across ordered sequences.

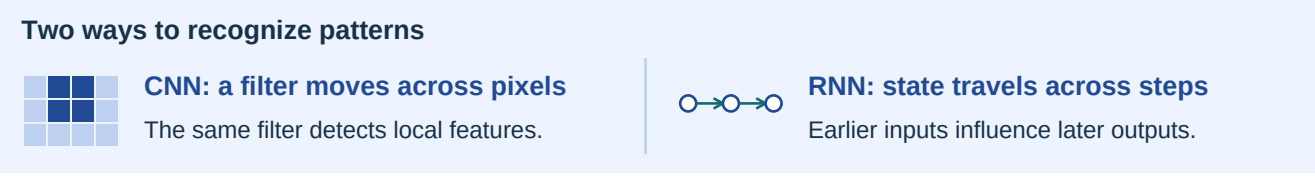
DEFINITION — Long Short-Term Memory (LSTM): A specialized RNN variant that uses internal cell states and gating mechanisms to preserve long-range sequence context reliably.

DEFINITION — Hidden State: A memory vector carried forward across sequence steps in a recurrent model to store contextual history.

In recent years, researchers created **Generative Models** and **Diffusion Models**. Rather than simply classifying images into categories, these advanced networks learn to synthesize entirely new synthetic image samples from scratch by reversing gradual noise steps.

DEFINITION — Generative Model: A neural network model designed to synthesize realistic new data samples based on statistical patterns learned from training data.

DEFINITION — Diffusion Model: A generative deep learning model trained to create structured images or audio by learning to systematically remove added random noise.



Part 3: How a Neural Network Learns

Having examined how neural network architectures are constructed, we now address the central question: how does an un-trained neural network adjust its millions of internal parameters to actually become smart?

This parameter adjustment process is called **training**. In the vast majority of practical deployment scenarios, models learn through a framework known as **supervised learning**.

In supervised learning, every training example consists of two distinct components: the input feature vector (such as an image of handwritten digit 5) and an authoritative answer called the **target reference label**.

> **DEFINITION — Supervised Learning:** A machine learning framework where network parameters are trained using paired inputs and verified reference target labels.

> **DEFINITION — Target Label (Reference Ground Truth):** The verified correct answer paired with an input feature during supervised training routines.

To evaluate a network properly during development, engineers divide available checked datasets into three completely separate splits: the **training set**, the **validation set**, and the **test set**.

Specifically, the network uses the training set directly to calculate gradient signals and update weights. Meanwhile, the validation set is used during development to compare hyperparameter choices and detect overfitting.

Finally, the test set remains strictly locked away until development finishes completely. It serves as an unbiased final check of real-world performance on previously unseen data.

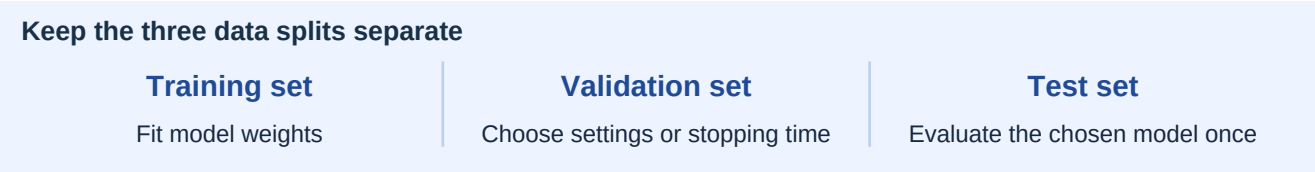
DEFINITION — Training Set: Examples used to fit model parameters. Backpropagation computes gradients. The optimizer updates the weights.

DEFINITION — Validation Set: A separate dataset split evaluated during development to tune hyperparameters and monitor general performance.

DEFINITION — Test Set: An isolated dataset split evaluated exclusively at the end of development to gauge final unbiased generalization capacity.

Crucially, mixing data across these splits leads to a severe error called **data leakage**. If identical image files cross between splits, test scores become deceptively high because the model simply memorized familiar examples.

NOTE — The Splitting Golden Rule: Never allow test set data to influence training steps or hyperparameter selection. Doing so turns the test set into developmental training material and destroys evaluation integrity.



When an input passes forward through a neural network, it calculates intermediate layer numbers until the final layer produces an output score called a **prediction**.

Importantly, a model's prediction comes strictly from calculated numbers, while the target reference label comes from checked ground-truth records. In early training iterations, predictions often disagree wildly with target labels.

DEFINITION — Prediction: The estimated output score or category generated by a neural network model when evaluating an input.

DEFINITION — Forward Pass (Forward Propagation): The step-by-step mathematical calculation process moving inputs forward through layers to yield an output prediction.

To measure exactly how wrong a prediction is, software evaluates a mathematical rule known as a **loss function**. The single scalar number produced by this rule is called the **loss**.

When predicting continuous numerical scalar outputs, models often use **squared-error loss**, which calculates the squared mathematical difference between the predicted number and the target value.

On the other hand, for multi-class classification tasks like identifying handwritten digits, models use **cross-entropy loss**, which penalizes models that assign low probabilities to correct target classes.

DEFINITION — Loss Function: A mathematical equation measuring the exact numerical mismatch between a model's calculated output prediction and true reference label.

DEFINITION — Loss: The single scalar metric emitted by a loss function indicating the degree of model error on given examples.

DEFINITION — Squared Error: A regression loss metric calculated by squaring the difference between predicted values and reference target values.

DEFINITION — Cross-Entropy Loss: A classification loss function measuring discrepancies between predicted probability scores and ground-truth target classes.

While loss calculates current error, it does not directly explain how individual weight parameters should be tweaked. To find out how to adjust each weight, the system computes **gradients**.

Mathematically, a gradient indicates how sensitive total loss is to tiny adjustments in a specific parameter, pointing in the direction of steepest loss increase.

To calculate gradients efficiently across millions of interconnected parameters, networks use an algorithm called **backpropagation**, which applies the calculus chain rule backward from the output layer to the input layer.

> **DEFINITION — Gradient:** A mathematical vector collecting partial derivatives that describe the magnitude and direction of loss changes relative to model parameters.

> **DEFINITION — Backpropagation:** An algorithm that calculates loss gradients across network layers by working backward from output errors using the calculus chain rule.

To understand the difference clearly, remember this rule: forward propagation calculates network output predictions, while backpropagation calculates gradient directions backward through the network.

Once backpropagation evaluates gradients, an algorithm called an **optimizer** steps in to execute parameter weight updates.

Gradient descent subtracts the learning rate times the gradient from each weight. A positive gradient lowers the weight. A negative gradient raises it. A zero gradient leaves the weight unchanged.

During optimization, engineers set a crucial tuning scalar called the **learning rate**, which dictates how large a step parameters take toward lower loss during each update iteration.

> **DEFINITION — Optimizer:** The mathematical algorithm responsible for executing parameter weight updates based on calculated gradients.

DEFINITION — Gradient Descent: An optimization method that iteratively updates parameters in the direction opposite to their gradients to reach minimum loss.

DEFINITION — Learning Rate: A hyperparameter scalar controlling the step size applied to parameter updates during gradient descent steps.

Setting the learning rate requires careful balance. If the learning rate is set too low, parameter updates move extremely slowly, causing training to take days or weeks.

Conversely, setting the learning rate too high causes parameter updates to overshoot optimal values wildly, making loss fluctuate chaotically or diverge completely.



During practical training routines, software rarely processes an entire dataset all at once. Instead, examples are grouped into small training blocks called **batches**.

Processing one single batch and executing one parameter update step is called an **iteration**. Furthermore, completing one full pass through every single training batch in the dataset defines an **epoch**.

Finally, after training completes, the model enters **inference** mode, where it evaluates brand-new real-world inputs to produce useful predictions without executing any further weight parameter updates.

DEFINITION — Batch: A subset of training examples grouped together for simultaneous forward processing and weight updating.

DEFINITION — Iteration: A single training step consisting of forward calculation, loss calculation, backpropagation, and parameter updating on one batch.

DEFINITION — Epoch: One complete execution cycle passing an entire training dataset forward and backward through a model once.

DEFINITION — Inference: Using an already-trained model to evaluate new input data and produce predictions without updating parameter weights.

However, achieving high performance on training data alone does not guarantee a successful model. The true goal of training is achieving strong **generalisation** on unseen data.

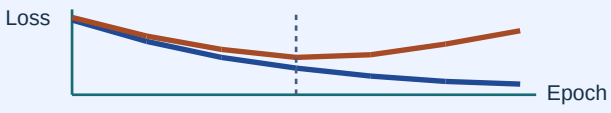
A common failure mode during training is **overfitting**, which occurs when a network memorizes noise, background quirks, and tiny details in training files rather than learning general patterns.

When a model overfits, its training loss score drops near zero, but its validation loss climbs sharply upward. To catch overfitting early, engineers track validation scores closely across epochs.

DEFINITION — Generalisation: The operational capability of a trained model to generate accurate predictions on new, previously unseen evaluation data.

DEFINITION — Overfitting: A training failure where a model memorizes specific training noise, degrading real-world performance on new data.

Reading a learning curve



Training loss **Validation loss**

After the dotted line, validation loss rises while training loss keeps falling.

Finally, we must emphasize that advanced algorithms cannot fix underlying data flaws. If training labels are incorrect or biased, supervised learning faithfully learns those exact errors.

Furthermore, model size alone does not replace solid data quality. A giant neural network trained on poor or unrepresentative data simply becomes very good at making confident mistakes.

Ultimately, successful Connectionist AI requires linking robust Big Data preparation pipelines with well-chosen network architectures, disciplined training routines, and continuous ethical oversight.

Part 4 & Conclusion: AI & Big Data Synthesis

Having explored Big Data infrastructure in Part 1 and Neural Network mechanics in Parts 2 & 3, we now smoothly bring these complementary pillars together. Modern technological solutions depend on connecting continuous data streams to intelligent connectionist models efficiently and ethically.

Part 4: Real-World Applications & Modern Implementations

In real-world industry settings, neural network architectures are not generic tools; they are specifically matched to distinct data formats and operational tasks. Connecting the right data pipeline to the appropriate model architecture ensures fast, reliable results.

For spatial vision tasks like medical imaging or visual quality checks, systems match **Convolutional Neural Networks (CNNs)** and **Diffusion Models** with high-resolution image data streams, as documented by **LeCun et al. (1998)** and **Ho et al. (2020)**.

Meanwhile, for continuous temporal streams—such as speech recognition, machinery sensor logs, or translation—systems rely on **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** architectures **(Hochreiter & Schmidhuber, 1997)**.

DEFINITION — Spatial & Temporal Matching:
• **Spatial Matching:** Pairing grid-structured images with CNN convolution operations to detect edges and textures.
• **Temporal Matching:** Pairing sequential audio or text streams with RNN/LSTM memory gates to retain context over time.

Matching an architecture to a task

Data	Architecture	Example task
Factory camera images	CNN	Detect a surface defect
Sensor readings over time	LSTM	Forecast machine temperature

1. Executive Summary & Conclusion

Ultimately, modern computing relies on a complete, interconnected journey: gathering massive amounts of raw data, organizing it using physical and cloud infrastructure, processing it with smart neural network architectures, and guiding those networks to learn through controlled training cycles.

First of all, everything starts with Big Data. Every digital action—whether sending a message, placing an online order, recording a sensor reading, or tracking a location—creates a data point characterized by Volume, Velocity, Variety, Veracity, and Value **(IBM Cloud Education, 2023)**.

Secondly, collecting and storing data is only the first half of the equation; systems must also find meaningful patterns. Connectionist AI addresses complex perceptual tasks using artificial neural networks inspired by biological nervous systems **(Stanford Encyclopedia of Philosophy, 2019)**.

Thirdly, neural networks do not begin with knowledge; they must be trained through supervised learning. By utilizing forward passes, loss functions, backpropagation gradients, and gradient descent optimizers, models continuously refine their parameters **(Google ML Crash Course, 2023)**.

Finally, technical accuracy alone is insufficient for real-world deployment. AI systems can inherit and amplify systemic distortions present in historical data. Therefore, robust ethical boundaries—including Data Minimisation, Transparency, Fairness, and Human Oversight—must guide every deployment **(NIST, 2023)**.

> **NOTE — Responsible AI Principles:** High technical accuracy cannot replace ethical governance. Systems must maintain human oversight, protect individual privacy, and prevent algorithmic bias across demographic groups.

2. References & Attribution Index

- **IBM Cloud Education (2023)** — *What is Big Data?* 5 V's definitions and scale thresholds.
- **Stanford Encyclopedia of Philosophy (2019)** — *Connectionism*. Conceptual sub-symbolic representations.
- **LeCun, Y. et al. (1998)** — *Gradient-Based Learning Applied to Document Recognition*. CNN spatial hierarchies.
- **Hochreiter, S. & Schmidhuber, J. (1997)** — *Long Short-Term Memory*. LSTM sequence gating mechanics.
- **Ho, J. et al. (2020)** — *Denoising Diffusion Probabilistic Models*. Reverse-step generative synthesis.
- **Google Machine Learning Crash Course (2023)** — *Gradient Descent & Optimization*. Parameter update dynamics.
- **NIST (2023)** — *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Ethics, bias, and oversight.

3. Master Glossary (Compact View)

Term	Domain	Concise Definition
Activation Function	Connectionist AI	Mathematical function applied to weighted sums to introduce non-linearity.
Artificial Neural Network	Connectionist AI	Computational model of interconnected neurons transforming numerical inputs into outputs.
Backpropagation	NN Training	Algorithm calculating loss gradients by stepping backward through network layers.
Batch Processing	Big Data	Processing a collected group of data records together as a single scheduled job.
Bias (Neuron Parameter)	Connectionist AI	Adjustable parameter added to weighted inputs prior to activation.
Big Data	Big Data	Data whose volume, velocity, and variety require specialized processing methods.
Cloud Storage	Infrastructure	Provider-managed infrastructure offering network-accessible storage.
Convolutional Network (CNN)	Connectionist AI	ANN architecture applying sliding spatial filters to grid inputs like images.
Cross-Entropy	NN Training	Loss function measuring divergence between predicted probabilities and target labels.

Term	Domain	Concise Definition
Data Minimisation	Ethics	Principle of collecting and retaining only data strictly necessary for a specified purpose.
Epoch	NN Training	One complete execution pass through an entire training dataset.
Gradient Descent	NN Training	Optimization method updating parameters opposite to calculated loss gradients.
Inference	NN Training	Evaluating real-time inputs using a trained model without parameter updates.
Learning Rate	NN Training	Scalar hyperparameter determining step size during optimization updates.
Long Short-Term Memory (LSTM)	Connectionist AI	Specialized RNN using cell state gates to preserve long-range sequence context.
Loss Function	NN Training	Mathematical metric measuring discrepancy between model outputs and targets.
Overfitting	NN Training	Training failure where a model memorizes noise, degrading generalisation on new data.
Stream Processing	Big Data	Continuous real-time ingestion and computation on incoming streaming data records.
Supervised Learning	NN Training	Training framework pairing input feature vectors directly with verified target labels.
Veracity	5 V's	Degree of accuracy, fidelity, completeness, and trust inherent in raw data.

A / BIG DATA

Cleaning a scan log

Goal

Prepare one reliable record per scan and explain which records need review.

Scan log

Each entry lists a scan ID, writer, digit label and pixel count. The scanner should return 784 pixels and one digit label from 0 to 9.

A17 Kim 5 784
A17 Kim 5 784 (exact copy of the same scan)
B04 Lee ? 784
C09 Jo 3 784
D11 Sam 8 780

Your steps

1. Mark each row as keep, duplicate or review.
2. Count the usable scans and list their IDs.
3. Decide how you would handle each record marked for review.

Expected result

A clean list with unique IDs, plus a reason for each excluded row.

Reflect

If the two A17 rows went into different training and test sets, how would that affect your confidence in the test score? Check p. 38.

Calculating a neuron output

Goal

Trace how a bias changes the response of one neuron to the same pair of pixels.

Inputs

The normalized pixel values are 0.8 and 0.2. Their weights are 0.5 and −0.25. Start with a bias of 0.10.

Your steps

1. Multiply each input by its weight.
2. Add both products and the bias to get the weighted sum z.
3. Apply ReLU: keep a positive z, and return zero otherwise.
4. Repeat with a bias of −0.50 and the same pixels and weights.

Expected result

Two weighted sums and two activation values. Keep the sign of each product visible in your working.

Reflect

How can the same two pixels activate the neuron in one calculation and leave it inactive in the other? Check p. 38.

C / TRAINING AND APPLICATIONS

Reading a handwritten digit

Goal

Trace a toy image to a prediction, then distinguish training from evaluation.

Example

A tiny crop from a handwritten 5 has two rows of pixel values: [0, 255] and [128, 0]. The checked label for the whole image is 5. For this exercise, a classifier gives 3 a score of 0.60 and 5 a score of 0.40.

Your steps

1. Read the crop row by row and divide each value by 255.
2. Choose the higher-scoring digit. Compare it with the checked label.
3. In a training example, suppose a weight is 0.40, its loss gradient is −0.30 and the learning rate is 0.10. Subtract the scaled gradient to update the weight.

Expected result

A four-number input vector, a prediction, the target label and one updated weight. The scores and gradient are supplied teaching values.

Reflect

If this were a held-out test image, would you update the weight? What should happen to a parcel whose digit is uncertain? Check p. 38.

PRACTICE / ANSWER KEY

Worked answers

A / Cleaning a scan log

Keep one A17 and C09: two usable scans. Drop the exact A17 copy. Verify B04 against the original label before admitting it. Check the original image or rescan D11 to recover the missing pixels. A duplicate in both splits lets the model see a test example during training and can inflate the score.

B / Calculating a neuron output

The weighted inputs are $0.8 \times 0.5 = 0.40$ and $0.2 \times (-0.25) = -0.05$. With bias 0.10, $z = 0.45$ and ReLU returns 0.45. With bias −0.50, $z = -0.15$ and ReLU returns 0. The bias moves the sum across the activation threshold.

C / Reading a handwritten digit

The input vector is [0, 1, 0.502, 0], rounded to three decimals. The prediction is 3 because 0.60 is the higher score. The label is 5, so this prediction is wrong. During training, backpropagation computes gradients and the optimizer updates weights. Here the new weight is $0.40 - 0.10 \times (-0.30) = 0.43$.

When the image is held out

Record the error and leave the weights fixed. Updating them on the test image would contaminate the evaluation. For an uncertain parcel, have a reviewer inspect the original label before routing it.